CS-107 Lecture 19
Introduction to Shell

Troy Astarte

Megan Venn-Wycherley

Goals for Today

- The Unix file system: directories, absolute and relative paths, hidden files
- Shells and the kernel: what a shell does, sh and bash, and why bash is the default
- The command line against the GUI: the prompt and your starting directory
- Command structure and getting help: man, --help, built-in commands and external executables
- Basic commands in use: touch, ls and cat, demonstrated live, then scripting

The Unix System

- Unix based OSs are a family of Operating System that utilise the hierarchical UNIX file system.
- Developed initially by AT&T for internal use, but it gradually spread and became more comprehensive.
- Not to be confused with a Linux kernel, which is derived from Unix but with key differences.
- Linux is an open source re-implementation of a Unix system and does not explicitly contain an OS, although distributions such as Ubuntu/OpenSUSE etc package kernel with OS.

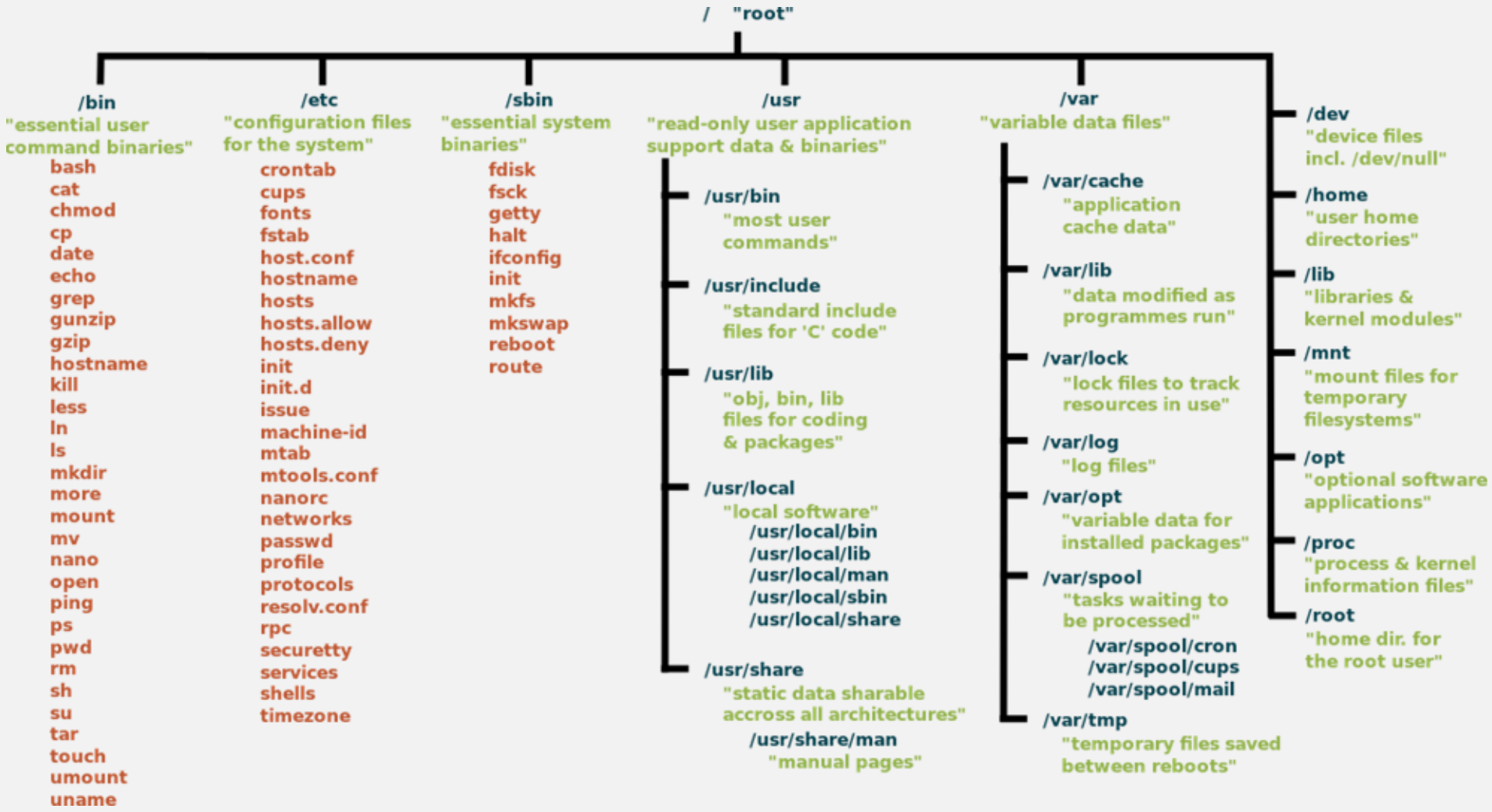
The Unix System

- A vast portion of the Unix and Unix-like system treats OS behaviours as files and manipulations of such files.

- A small number of file types are available, including:
  - Text files
  - Directories
  - Symbolic links
  - FIFO (pipes, see later)
  - …

- Many **commands** defined which interact with such file types

The Unix System



The Unix System

- As covered in previous lectures, the directory is just a special file type
  - Contains the information about all files below it.
  - A directory points to it's parent (..) and itself (.)
- A file can be accessed by using it's **relative** or **absolute** pathname
  - Absolute path describes the **path from the root**
  - Relative path describes the path in **relation to the current directory**
- Some files can be indicated as **"hidden"** by prepending their name with a period.
  - **MyFile.txt** is not hidden, but **.MyFile.txt** is.

Definition of **Shell**

- …an outer layer of an Operating System.
- …an interface between the user and the internal parts of the operating system (or the Kernel)…

[http://www.linfo.org/shell.html]

What is the Kernel?

- The kernel talks to the hardware, software and manages the systems resources (RAM, CPU, BUSes).
- A kernel is at the heart of Mac OS, Windows & Linux.
- It's the core of the operating system and must run reliably and efficiently for the computer system to work correctly, prevent data and processes corrupting and to re-use resources efficiently.
- Linux is open source, you can download and compile your own Linux kernel if you can use the command line.

Shells

- A number of different shells have been developed for Unix-like operating systems.
- They share many similarities, but there are also some differences with regard to commands, syntax and functions that are important mainly for advanced users.
- Every Unix-like operating system has at least one built-in shell, and most have several.

http://www.linfo.org/shell.html

Shell Prompt or Command Line

- A ***shell prompt***, also referred to as a command prompt is a character or set of characters at the start of the ***command line*** that indicates that the shell is ready to receive commands.

- It is usually denoted with a dollar sign ($) for ordinary users and a pound sign (#) for the root (i.e., administrative) user.

- The term ***command line*** is sometimes used interchangeably with the ***shell prompt***, because that is where the user enters commands.

http://www.linfo.org/shell.html

The **sh** Shell

- **sh** (the Bourne Shell) is the original UNIX shell, and it is still in widespread use today.

- Written by Stephen Bourne at Bell Labs in 1974, it is a simple shell with a small size and few features, perhaps the fewest of any shell for a Unix-like operating system.

- Bell Labs was the research and development arm of AT&T (The American Telephone and Telegraph Company). The first version of UNIX was developed at Bell Labs in 1969.

http://www.linfo.org/shell.html

The ***bash*** Shell

- ***bash*** (Bourne-again shell) is the default shell on Linux.
- Runs on nearly every other Unix-like operating system as well, versions are also available for other operating systems, including Windows.
- ***Bash*** is a superset of ***sh*** (i.e., commands that work in ***sh*** also work in ***bash***, but the reverse is not always true), and it has many more commands than ***sh***, making it a powerful tool for advanced users.

http://www.linfo.org/shell.html

Why ***bash***?

- It is intuitive and flexible, and thus it is probably the most suitable shell for beginners.

- ***bash*** was written for the GNU project (whose goal is to develop a complete, Unix-compatible, high performance and entirely free operating system), primarily by Brian Fox and Chet Ramey.

- Its name is a pun on the name of Steve Bourne.

http://www.linfo.org/shell.html

Evolution of the Interface

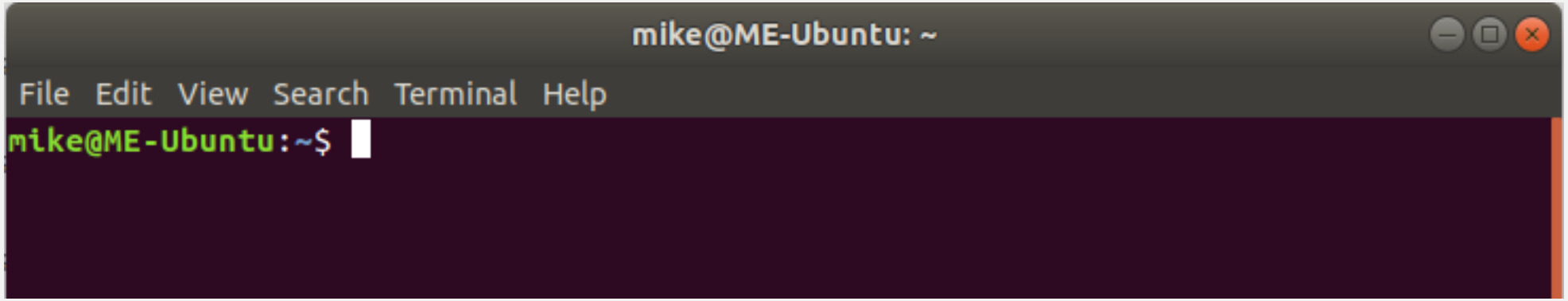
- User Interface was carried out using a series of commands executed in a batch, often fed in using punch cards or tape.
- These become known as ‘interfaces’.
- Replaced by batch files or scripts today and run from the command line or shell.

GUI & CLI

- GUI (Graphical User Interface) provides a visual user experience and needs a navigation device such as a mouse or touch screen (although you can tab around some options).
- If you are used to using a mouse or a touch screen and are familiar with looking at menus and icons a GUI will let the user explore an OS's functionality without the need to know key command words or any command syntax.
- CLI (Command Line Interface) uses a text interface only.
- The CLI can be connected to with just a keyboard and text output, allowing access to a computer's OS interface without needing a screen.

Starting directory

- Each user account will have a **home directory.**
- Contains the user's personal files and information.
- When logging into a shell, it starts within the **starting directory.**
  - This is often the user's home directory by default.



Commands and Applications

- A shell is a command-line interpreter, it is designed to execute programs called **commands**.
- It can also execute larger programs, applications and user-defined code.
- Commands come in the following form:

$ command [arg1] [arg2] …[argN]

Where $ is the command prompt, and [arg] indicates a number of optional arguments passed to the command.

Commands and Applications

- The Unix OS family is known for being terse (concise)
- Commands can often be short and cryptic, with little use of vowels
  - ls: **L**i**s**t
  - cd: **C**hange **D**irectory
  - grep: **G**lobally search a **R**egular **E**xpression and **P**rint
- Not always helpful / memorable. But you'll become proficient with practice.

Commands and Applications

- Many commands have optional flags or option switches.
- In the same vein as being concise, and avoiding a large number of very similar commands with slightly varied behaviour.
- Flags are often (but not always!) denoted with a - or -- notation and used to indicate functionality to carry out.
- Although they have this special behaviour, they are just inputs passed to the command as seen in the last slide.

Commands and Applications

- Consider the command **ls**:

  $ ls **-l -R** ./Documents

  $ ls **-R -l** ./Documents

  $ ls **-lR** ./Documents

-l (lowercase L) provides a "long listing" or more detail

-R tells the command to recursively list the contents of subdirectories

- The ordering of the flags does not matter here, all of these do the same thing.
- Convention is to have the flags before the arguments of the command:

  $ command flag1 flag2 … flagN arg1 arg2 … argN

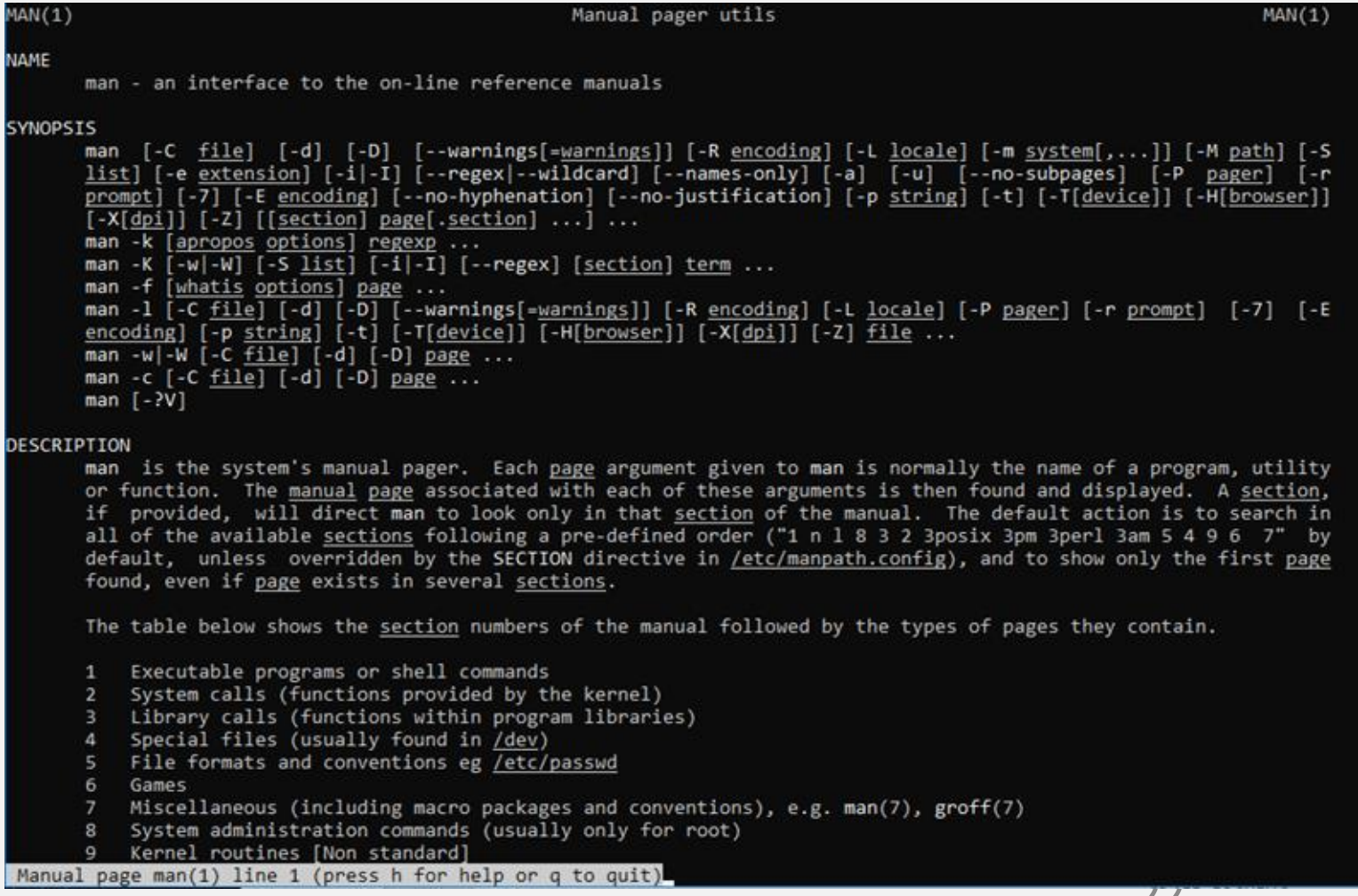
Commands and Applications

- But how to find out about these commands and their options?
- Luckily someone has (usually) taken the time to write documentation
- We can access documentation for built-in commands with the *man* command.
- There is also usually a --help flag which will provide similar insight.

man

- https://ss64.com/bash/man.html
- Man displays help pages.
- For more information ***man man***

```
man command_name
```



-?, --help or -h

- Some commands offer help outside of using man.
- -? May give a clue as what to do.
- -h or -help should give good essential information on how to use a command or application.

e.g.

ls --help

```
        -x      List by lines
        -d      List directory entries instead of contents
        -L      Follow symlinks
        -H      Follow symlinks on command line
        -R      Recurse
        -p      Append / to dir entries
        -F      Append indicator (one of */=@|) to entries
        -l      Long listing format
        -i      List inode numbers
        -n      List numeric UIDs and GIDs instead of names
        -s      List allocated blocks
        -e      List full date and time
        -h      List sizes in human readable format (1K 243M 2G)
        -r      Sort in reverse order
        -S      Sort by size
        -X      Sort by extension
        -v      Sort by version
        -c      With -l: sort by ctime
        -t      With -l: sort by mtime
        -u      With -l: sort by atime
        -w N    Assume the terminal is N columns wide
        --color[={always,never,auto}]   Control coloring
[root@localhost ~]#
```

Built-in Commands vs External Executables

- A shell provides an environment in which we can run our available commands.
- In order to do so we require a number of built-in commands which ship with the shell
- These can vary per shell, but include the basics like:
  - cd - Changing directory
  - exec - Executing an external executable command
  - pwd - Print absolute path of current directory
- BASH also defines its own built-ins. Can you find which are BASH only?

Built-in Commands vs External Executables

- Built in commands such as **clear** (clear screen), **cd** (change directory) and **mkdir** (make directory) are present in the shell.

- Applications such as **nano** (a text editor), can be run in command line.

- Services such as **httpd** (apache web server), **ftp** (files transfer protocol) and **ssh** (secure shell) can be started and stopped using the Linux command line.

Some basic commands to get you going

mkdir - Make a directory

***mkdir mydirectory* -** make directory in current directory which is called "mydirectory"

cd - Change Directory

***cd mydirectory* -** change to "mydirectory" **iff** it exists

***cd* .. -** change directory into the parent directory

Some basic commands to get you going

ls - List directory contents

ls - list the current directory's visible contents

ls mydirectory - list visible contents of "mydirectory"

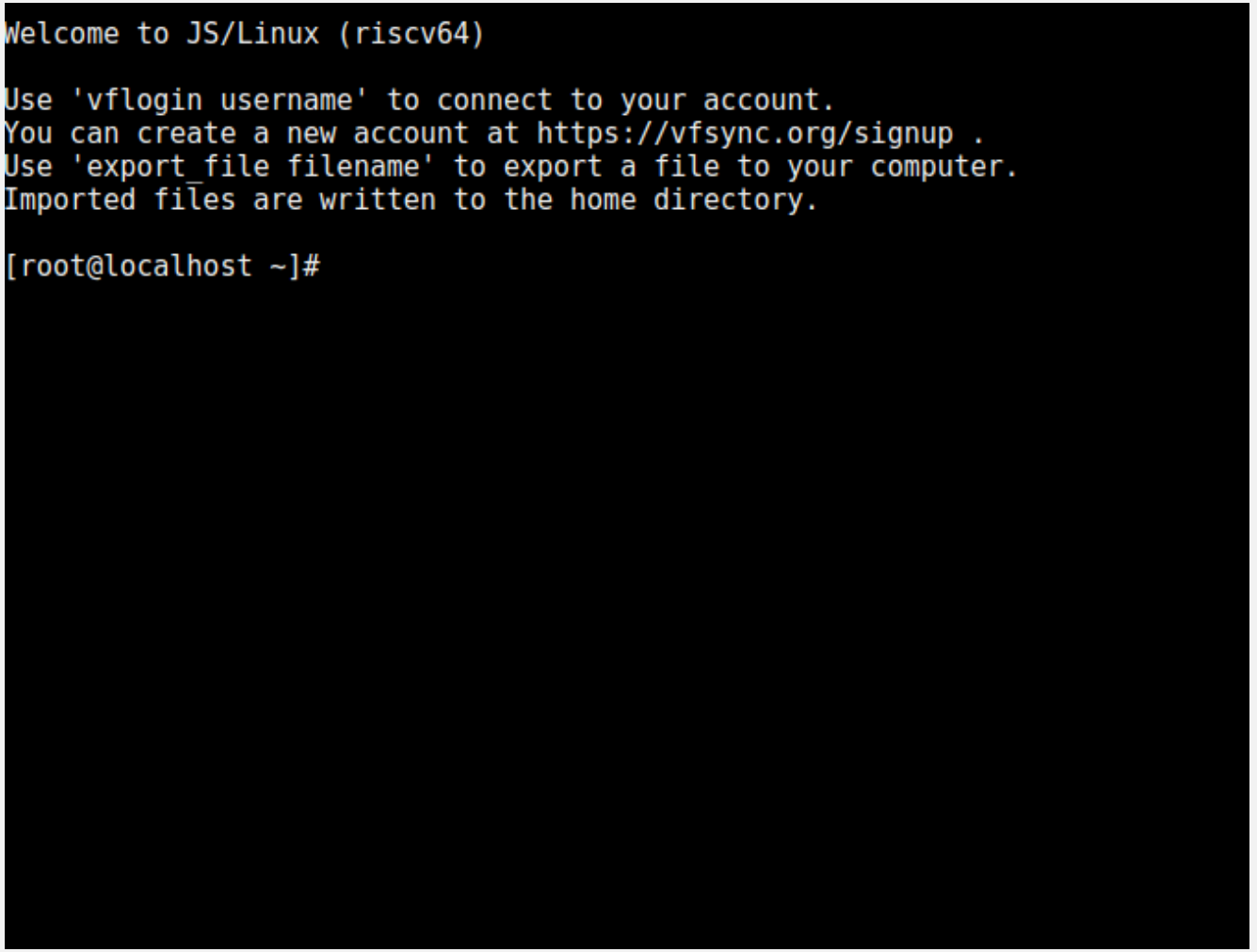
ls -a mydirectory - list visible & hidden contents of "mydirectory"

touch - Create file if it doesn't exist, or update last access time.

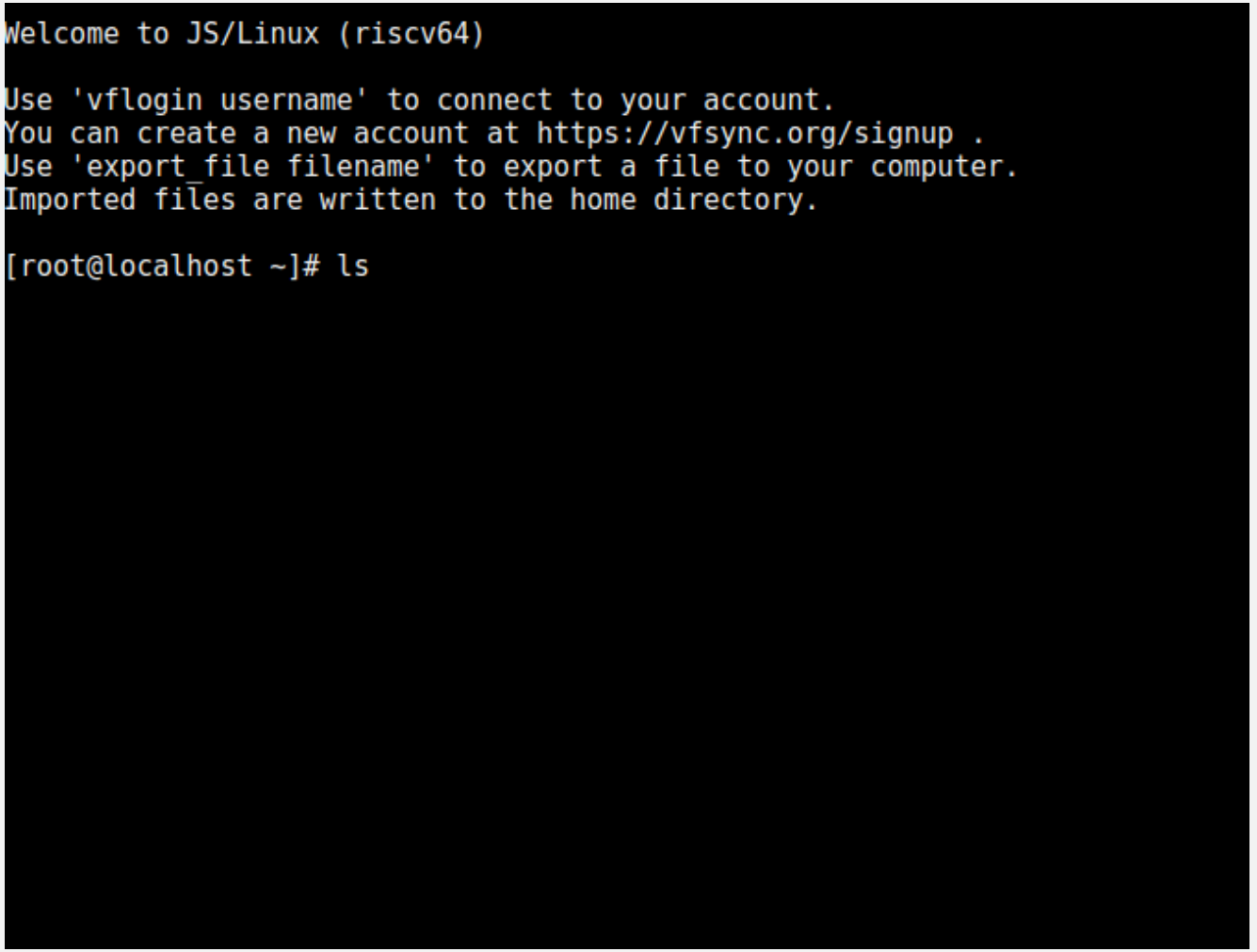
touch myfile - creates "myfile" file if it doesn't already exist

touch myfile - updates time last accessed

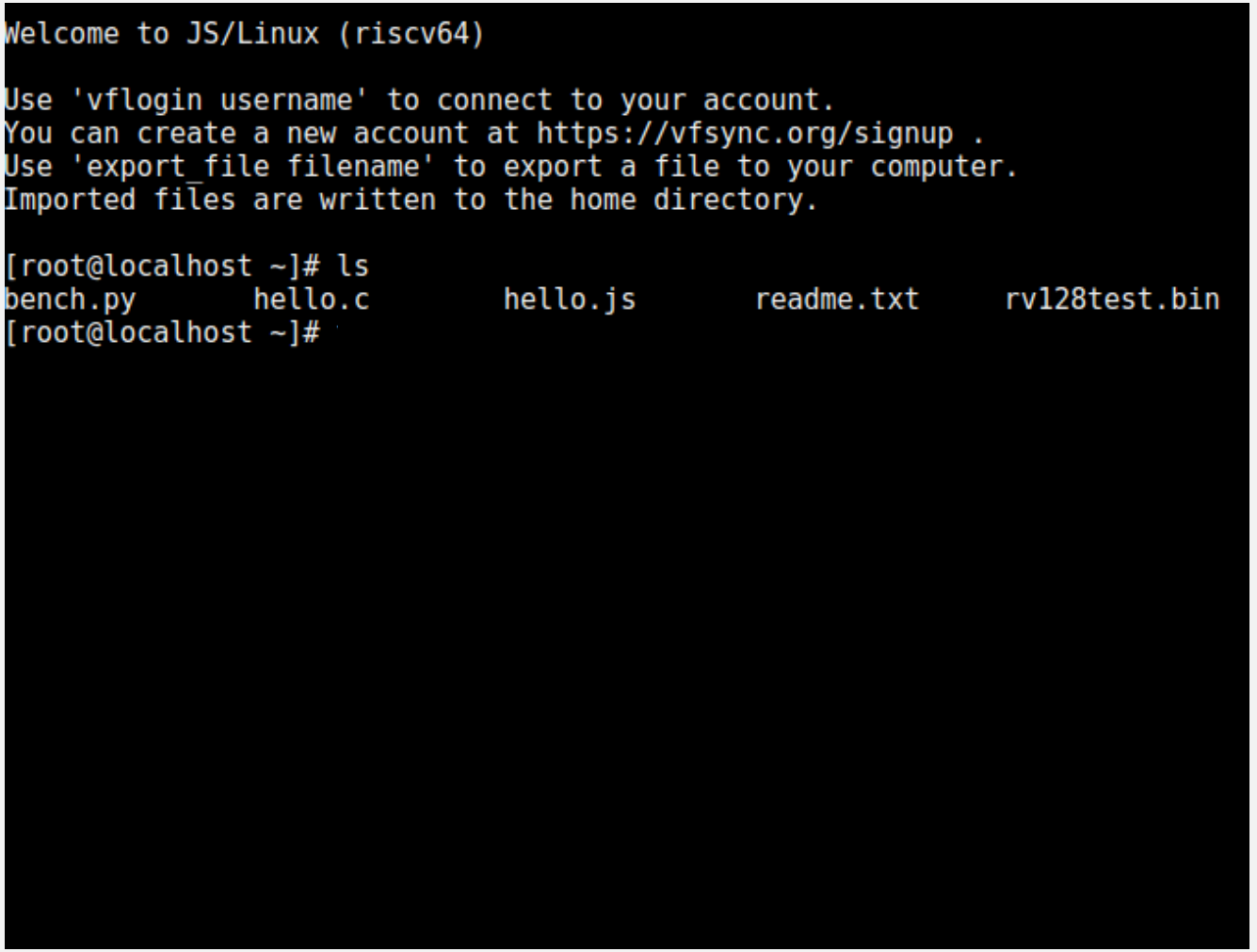
Shell demo: touch and ls



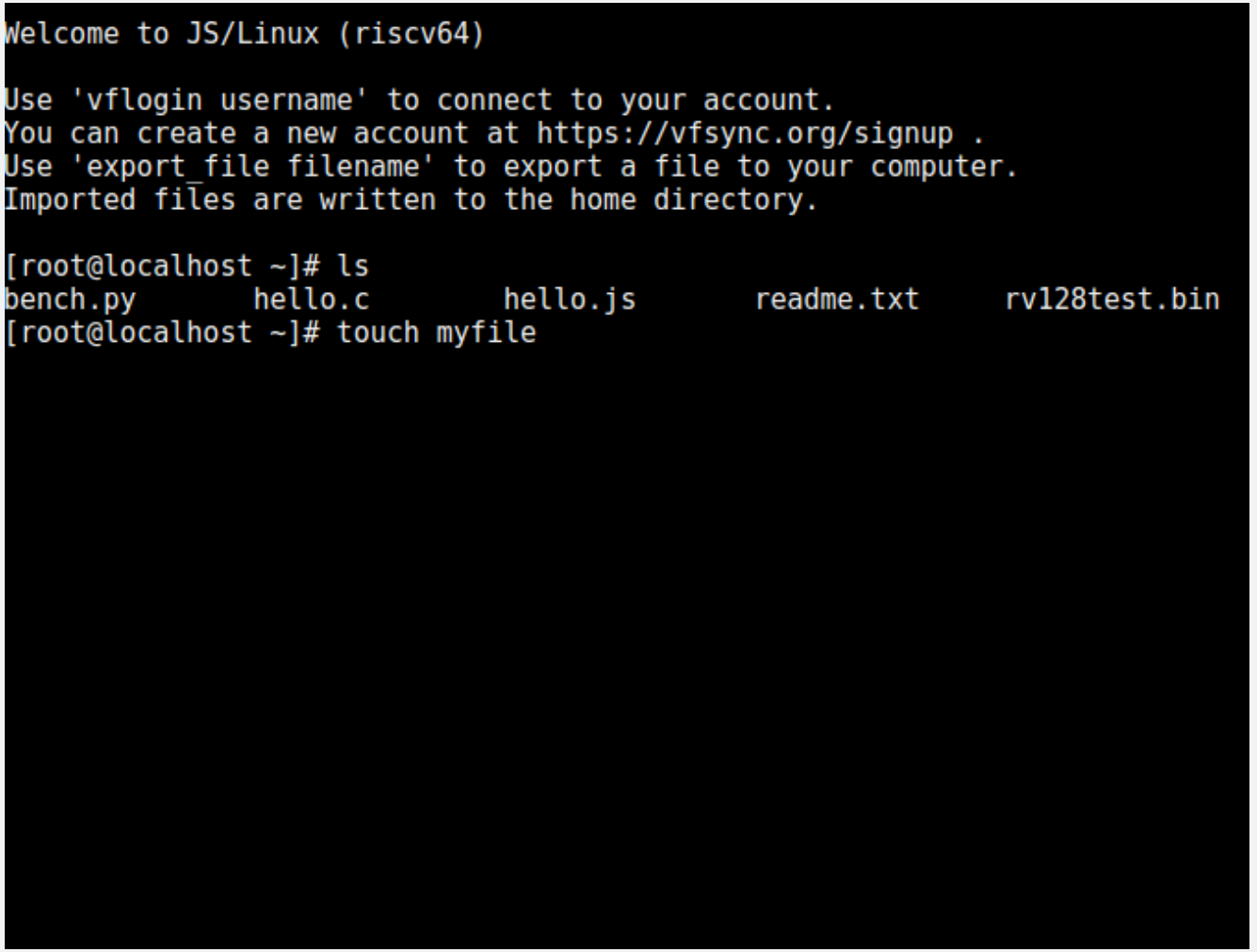
Shell demo: touch and ls



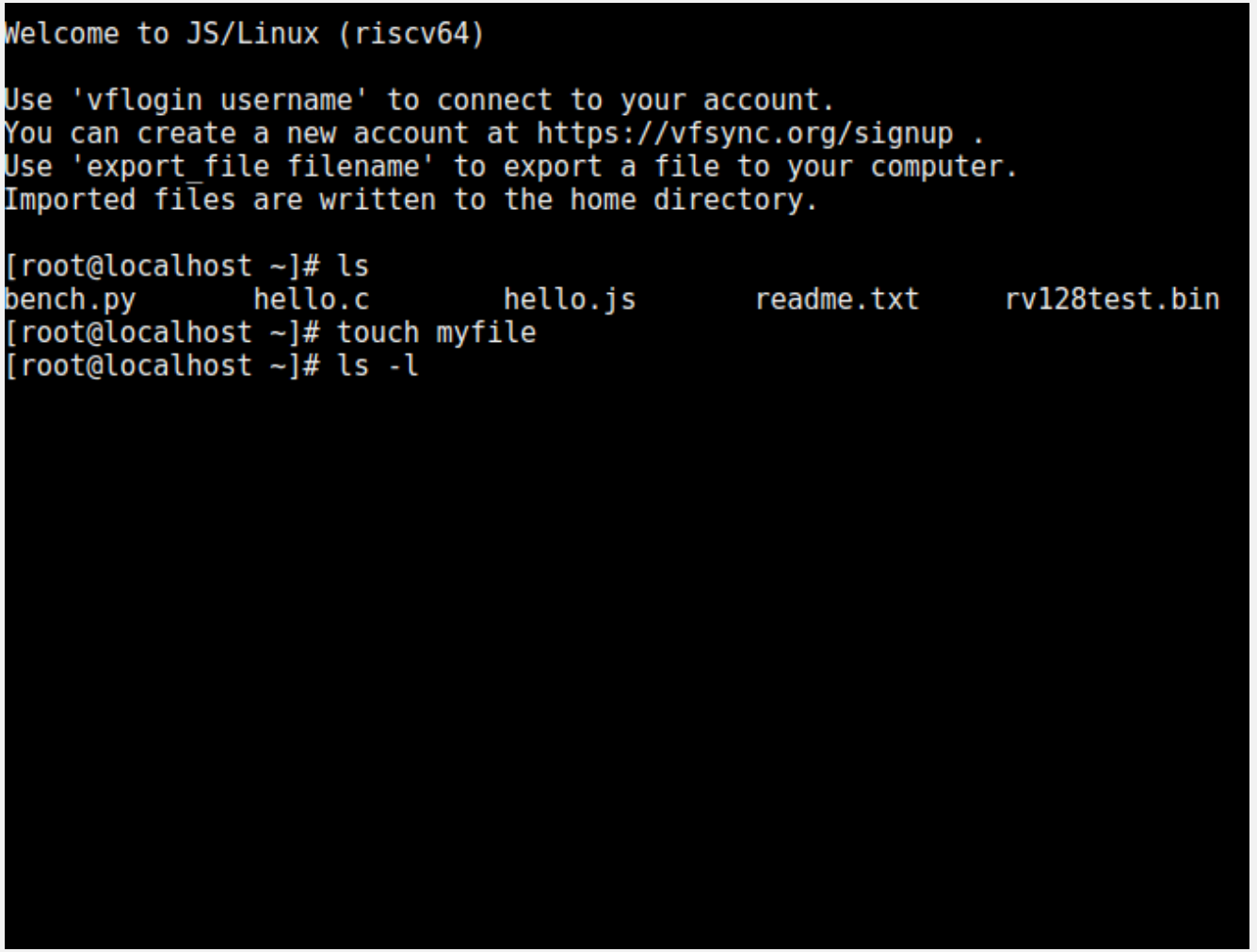
Shell demo: touch and ls



Shell demo: touch and ls



Shell demo: touch and ls



Shell demo: touch and ls

myfile is created!

```
Welcome to JS/Linux (riscv64)

Use 'vflogin username' to connect to your account.
You can create a new account at https://vfsync.org/signup .
Use 'export_file filename' to export a file to your computer.
Imported files are written to the home directory.

[root@localhost ~]# ls
bench.py       hello.c        hello.js       readme.txt     rv128test.bin
[root@localhost ~]# touch myfile
[root@localhost ~]# ls -l
total 28
-rw-r--r--    1 root     root           113 Sep  9  2018 bench.py
-rw-r--r--    1 root     root           185 Sep  9  2018 hello.c
-rw-r--r--    1 root     root            22 Sep 18 19:31 hello.js
-rw-r--r--    1 root     root             0 Mar 10 09:33 myfile
-rw-r--r--    1 root     root           238 Sep 18 19:38 readme.txt
-rw-r--r--    1 root     root          8256 Sep  9  2018 rv128test.bin
```

Shell demo: touch and ls

```
Welcome to JS/Linux (riscv64)

Use 'vflogin username' to connect to your account.
You can create a new account at https://vfsync.org/signup .
Use 'export_file filename' to export a file to your computer.
Imported files are written to the home directory.

[root@localhost ~]# ls
bench.py       hello.c        hello.js       readme.txt     rv128test.bin
[root@localhost ~]# touch myfile
[root@localhost ~]# ls -l
total 28
-rw-r--r--    1 root     root           113 Sep  9  2018 bench.py
-rw-r--r--    1 root     root           185 Sep  9  2018 hello.c
-rw-r--r--    1 root     root            22 Sep 18 19:31 hello.js
-rw-r--r--    1 root     root             0 Mar 10 09:33 myfile
-rw-r--r--    1 root     root           238 Sep 18 19:38 readme.txt
-rw-r--r--    1 root     root          8256 Sep  9  2018 rv128test.bin
[root@localhost ~]# touch myfile
```

Shell demo: touch and ls

```
Welcome to JS/Linux (riscv64)

Use 'vflogin username' to connect to your account.
You can create a new account at https://vfsync.org/signup .
Use 'export_file filename' to export a file to your computer.
Imported files are written to the home directory.

[root@localhost ~]# ls
bench.py       hello.c        hello.js       readme.txt     rv128test.bin
[root@localhost ~]# touch myfile
[root@localhost ~]# ls -l
total 28
-rw-r--r--    1 root     root           113 Sep  9  2018 bench.py
-rw-r--r--    1 root     root           185 Sep  9  2018 hello.c
-rw-r--r--    1 root     root            22 Sep 18 19:31 hello.js
-rw-r--r--    1 root     root             0 Mar 10 09:33 myfile
-rw-r--r--    1 root     root           238 Sep 18 19:38 readme.txt
-rw-r--r--    1 root     root          8256 Sep  9  2018 rv128test.bin
[root@localhost ~]# touch myfile
[root@localhost ~]# ls -l
```

Shell demo: touch and ls

```
Welcome to JS/Linux (riscv64)

Use 'vflogin username' to connect to your account.
You can create a new account at https://vfsync.org/signup .
Use 'export_file filename' to export a file to your computer.
Imported files are written to the home directory.

[root@localhost ~]# ls
bench.py       hello.c        hello.js       readme.txt     rv128test.bin
[root@localhost ~]# touch myfile
[root@localhost ~]# ls -l
total 28
-rw-r--r--    1 root     root           113 Sep  9  2018 bench.py
-rw-r--r--    1 root     root           185 Sep  9  2018 hello.c
-rw-r--r--    1 root     root            22 Sep 18 19:31 hello.js
-rw-r--r--    1 root     root             0 Mar 10 09:33 myfile
-rw-r--r--    1 root     root           238 Sep 18 19:38 readme.txt
-rw-r--r--    1 root     root          8256 Sep  9  2018 rv128test.bin
[root@localhost ~]# touch myfile
[root@localhost ~]# ls -l
total 28
-rw-r--r--    1 root     root           113 Sep  9  2018 bench.py
-rw-r--r--    1 root     root           185 Sep  9  2018 hello.c
-rw-r--r--    1 root     root            22 Sep 18 19:31 hello.js
-rw-r--r--    1 root     root             0 Mar 10 09:34 myfile
-rw-r--r--    1 root     root           238 Sep 18 19:38 readme.txt
-rw-r--r--    1 root     root          8256 Sep  9  2018 rv128test.bin
[root@localhost ~]#
```

Shell demo: touch and ls

```
Welcome to JS/Linux (riscv64)

Use 'vflogin username' to connect to your account.
You can create a new account at https://vfsync.org/signup .
Use 'export file filename' to export a file to your computer.
Imported files are written to the home directory.

[root@localhost ~]# ls
bench.py       hello.c        hello.js       readme.txt     rv128test.bin
[root@localhost ~]# touch myfile
[root@localhost ~]# ls -l
total 28
-rw-r--r--    1 root     root           113 Sep  9  2018 bench.py
-rw-r--r--    1 root     root           185 Sep  9  2018 hello.c
-rw-r--r--    1 root     root            22 Sep 18 19:31 hello.js
-rw-r--r--    1 root     root             0 Mar 10 09:33 myfile
-rw-r--r--    1 root     root           238 Sep 18 19:38 readme.txt
-rw-r--r--    1 root     root          8256 Sep  9  2018 rv128test.bin
[root@localhost ~]# touch myfile
[root@localhost ~]# ls -l
total 28
-rw-r--r--    1 root     root           113 Sep  9  2018 bench.py
-rw-r--r--    1 root     root           185 Sep  9  2018 hello.c
-rw-r--r--    1 root     root            22 Sep 18 19:31 hello.js
-rw-r--r--    1 root     root             0 Mar 10 09:34 myfile
-rw-r--r--    1 root     root           238 Sep 18 19:38 readme.txt
-rw-r--r--    1 root     root          8256 Sep  9  2018 rv128test.bin
[root@localhost ~]#
```

The timestamp on myfile is updated

Batch Commands or Scripting

- Command-line programming is powerful, but writing out a larger process sequentially every time can be exhausting.

- Often we may combine multiple command calls into a single script or batch.

- A series of shell commands executed from a file.

- Called **.bat** files in Windows type OS or **shell script** files in Unix / Linux.

How to Compile Your Own Linux Kernel
Austin Luong March 9, 2017

further reading:

https://www.makeuseof.com/tag/compile-linux-kernel/

Location UK	6 months to 8 Mar 2019	Same period 2018	Same period 2017
UK median annual salary	£52,500	£46,000	£42,500
Median salary % change year-on-year	+14.13%	+8.24%	-4.49%

Selected lines taken from on 8/3/2019
https://www.itjobswatch.co.uk/jobs/uk/linux%20command%20line.do

Comparative developer salaries (US).

Linux Kernel Developer	$162,595
Python Developer	$128,412
Swift Developer	$113,696
Java Developer	$105,888
Software Developer	$86,355
Web Developer	$72,229

From: ziprecruiter.com/Salaries Feb 2020

Comparative developer salaries.

Security Engineer	£60,000
Data Science Engineer	£70,000
Software Engineer	£55,000

From: IT Jobs Watch Jan 2020

Some ***bash*** Commands, in use.

- **The 10 Most Important Linux Commands** http://www.informit.com/blogs/blog.aspx?uk=The-10-Most-Important-Linux-Commands [Brad Yale, informit.com]

- **ls [list], cd [change directory], mv [move],**
- **man [manual],**
- **mkdir [make new directory], rmdir [remove directory],**
- **touch [make file], rm [remove file],**
- **locate [find file],**
- **clear [clears the screen].**

http://bit.ly/2VwoQsF

bash Reference Material

- https://ss64.com/bash/ : a handy alphabetical list of commands
- Directory Commands
- File Commands
- Redirection
- Common Flags
- Permissions
- Wild Cards
- Process Control



Useful links to Bash tutorials and explanations:

https://www.learnshell.org/

Interacting BASH scripting tool. (Not great in my opinion, but some interesting examples).

https://guide.bash.academy/inception/

Nice explanations of what BASH is and how it works.

https://linuxconfig.org/bash-scripting-tutorial-for-beginners

Some short videos of the tools working, less wordy than guide.bash.academy

https://ryanstutorials.net/bash-scripting-tutorial/

Some good recommendation online for this tutorial. Odd font for headings and several ads, but worth a look.

Goals for Today

- The Unix file system: directories, absolute and relative paths, hidden files
- Shells and the kernel: what a shell does, sh and bash, and why bash is the default
- The command line against the GUI: the prompt and your starting directory
- Command structure and getting help: man, --help, built-in commands and external executables
- Basic commands in use: touch, ls and cat, demonstrated live, then scripting

CS-107 Lecture 20
Working with Files

Troy Astarte

Megan Venn-Wycherley

Goals for Today

- Directory commands: ls, cd, mkdir, rmdir and pwd
- File commands: touch, cat, cp, mv, rm, head and less
- Writing to stdout with echo, and commenting your commands
- File permissions and ownership, and changing them with chmod and chown
- Redirection with >, >> and <, and joining commands with the pipe |

Directory Commands

- Directory Commands:
  - ls
  - cd
  - mkdir
  - rmdir
  - pwd

The list command: ls

https://ss64.com/bash/ls.html

List contents of current directory:-

```
ls .
```

List contents of the parent directory:-

```
ls ..
```

And this?

```
ls ../root
```

```
[root@localhost ~]# ls .
bench.py       hello.c        readme.txt     rv128test.bin
[root@localhost ~]# ls ..
bin      etc      lib      linuxrc  mnt      proc     run      sys      usr
dev      home     lib64    media    opt      root     sbin     tmp      var
[root@localhost ~]# ls ../root
bench.py       hello.c        readme.txt     rv128test.bin
[root@localhost ~]#
```

Change Directory: cd

https://ss64.com/bash/cd.html

- Change the current working directory to a specified folder.
- If directory is not specified, change to 'home' directory for user.

```
cd [options] [directory]
e.g.
    cd myfolder
    ls
    cd
    ls
```

```
[root@localhost ~]# cd myfolder
[root@localhost myfolder]# ls
[root@localhost myfolder]# cd
[root@localhost ~]# ls
bench.py       hello.c        myfolder       readme.txt     test
hello          long           q              rv128test.bin
[root@localhost ~]#
```

Make Directory: mkdir

https://ss64.com/bash/mkdir.html

- Create a new folder or folders unless they already exist.
- **`mkdir`** creates the standard entries . (dot) for the current folder and .. (dot dot) for its parent

`mkdir [`*`options`*`] `*`folder`*`…`

e.g.

```
mkdir "name with spaces"
mkdir name_with_no_spaces
```

```
[root@localhost ~]# mkdir "a folder"
[root@localhost ~]# ls
a folder        hello           long
bench.py        hello.c         myfolder
[root@localhost ~]#
```

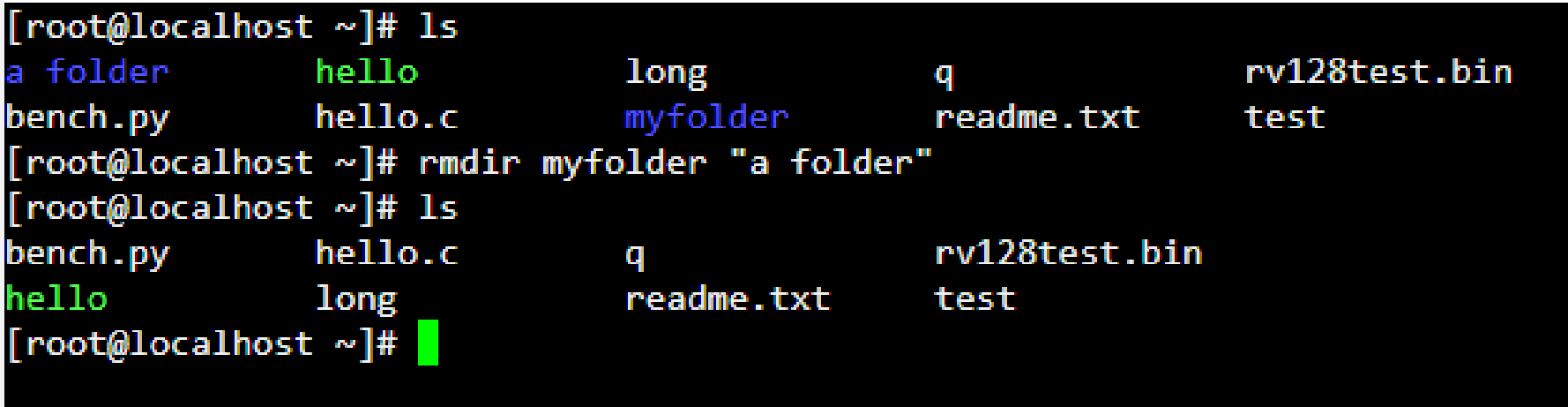
Remove Directory: rmdir

https://ss64.com/bash/rmdir.html

- Only works if the folders are empty

`rmdir [options] folder(s)...`

`e.g. rmdir myfolder "a folder"`



Print Working Directory: pwd

https://ss64.com/bash/pwd.html

- Options
  - `-P` :The pathname printed will not contain symbolic links.
  - `-L` :The pathname printed can contain symbolic links.
- The default action is to show the current folder as an absolute path

```
pwd [-LP]
```

e.g.

```
pwd
```

```
mike@ME-Ubuntu:~/Desktop/Repos/sushigo$ pwd
/home/mike/Desktop/Repos/sushigo
mike@ME-Ubuntu:~/Desktop/Repos/sushigo$
```

File Commands

- touch
- cat
- cp
- mv
- rm
- head
- less

Touch a file: touch

https://ss64.com/bash/touch.html

- Change file timestamps, change the access and/or modification times of the specified files
- Creates a file, if it does not exist or updates it to current timestamps if it does

```
touch [options] file(s)...
e.g.
      touch newfile
```

touch example

The file “newfile” is created

As newfile exists, this new call to touch just updates the timestamp



The **cat** Command

https://ss64.com/bash/cat.html

- The concatenate command.
  - Concatenates FILEs and prints them to **stdout.**
  - When file is '**-**' read standard input.

`cat [options] [filenames] [-] [file`

e.g.

```
cat readme.txt hello.c - > concatenated.txt
Hello! <ENTER>
Another line of text. <ENTER>
<CTRL>+D
```

```
[root@localhost ~]# cat concatenated.txt
Some tests:

- Compile hello.c:

  gcc hello.c -o hello
  ./hello

- Compute 1000 digits of pi:

  tinypi 1000 pi.txt
  cat pi.txt

- Run the 128 bit version of riscvemu:

  riscvemu128 -m 16 rv128test.bin
/* This C source can be compiled with:
   gcc -o hello hello.c
*/
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    printf("Hello World\n");
    return 0;
}
Hello!
Another line of text.
[root@localhost ~]#
```

Copy: cp

https://ss64.com/bash/cp.html

- Copy SOURCE to DEST, or multiple SOURCE(s) to DIRECTORY
- Copy one or more files.

```
cp [options] source destination
cp [options] source... directory
```

e.g.

```
cp -v readme.txt myfolder
```

```
[root@localhost ~]# cp -v readme.txt myfolder
'readme.txt' -> 'myfolder/readme.txt'
[root@localhost ~]# ls -l myfolder
total 4
-rw-r--r--    1 root     root           206 Feb 28 16:40 readme.txt
[root@localhost ~]#
```

Move: mv

https://ss64.com/bash/mv.html

- Move or rename files or directories
- If the last argument is an existing directory, **mv** moves the other given files into that directory
- If only two file names are given, it renames the first as the second

```
mv [options] source destination
mv [options] source... directory
```

e.g.

mv myfolder/readme.txt newreadme.txt

Remove: rm

https://ss64.com/bash/rm.html

- Removes or deletes files and links
- Does not remove directories without using `-r` option
- See **`shred`** for data safe deletion

`rm [`*`options`*`]` *`filename...`*

e.g.

```
rm newreadme.txt
```

Show head of files: head

https://ss64.com/bash/head.html

- Output the first part of files, prints the first part (10 lines by default) of each file

head [*options*] [*file(s)...*]

e.g.

head *.c *.txt

```
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char **argv)
{
    printf("Hello World\n");
    return 0;

==> readme.txt <==
Some tests:

- Compile hello.c:

  gcc hello.c -o hello
  ./hello

- Compute 1000 digits of pi:

  tinypi 1000 pi.txt
[root@localhost ~]#
```

Page through files: less

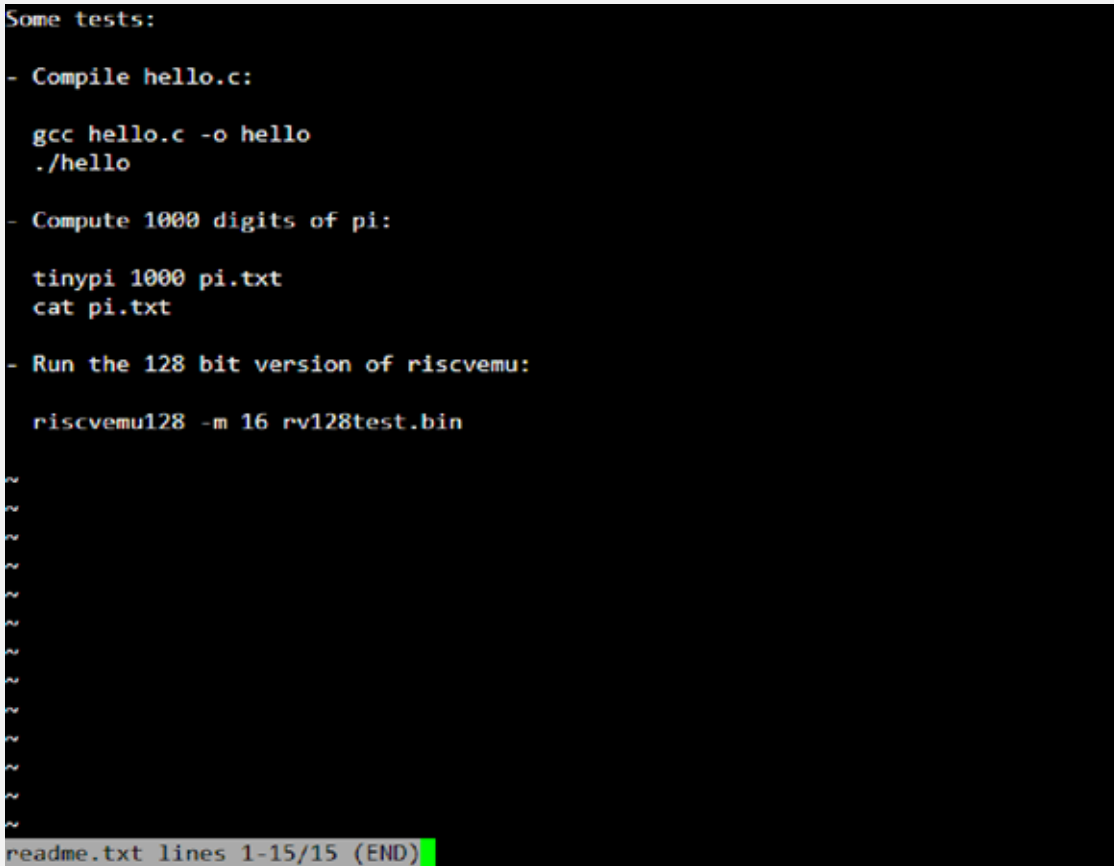
https://ss64.com/bash/less.html

- Page through text one screenful at a time, Search through output, Edit the command line.
- Very powerful on full bash implementations.

less [*options*] *file*

e.g.

```
less -M readme.txt
cat long | less -E
less -E long
less -?
```



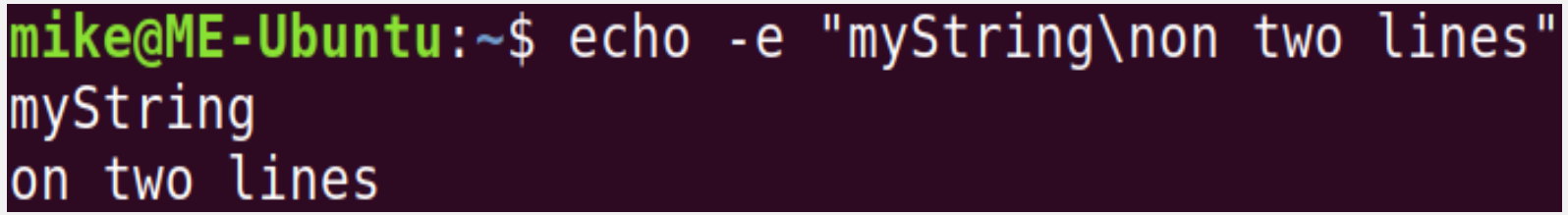
Echoing string to stdout: echo

- What if you still can't type multiple lines?
- You can try using "escape characters" with the -e option flag

e.g.

```
>> echo –e "myString\non two lines"

myString
on two lines
```



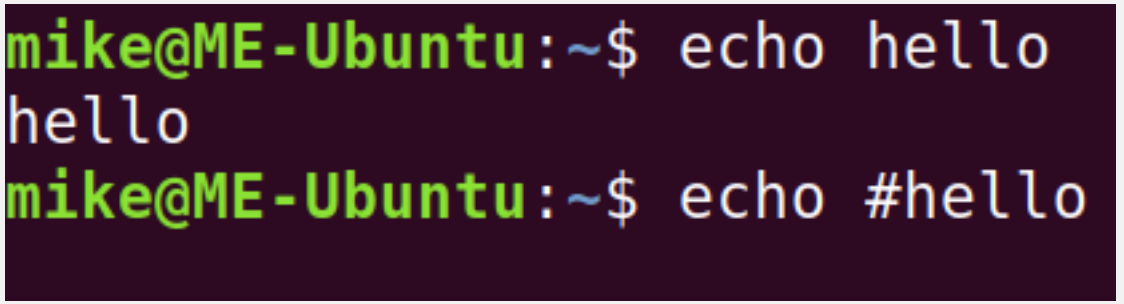
Commenting

- # symbol denotes rest of line as comment

e.g.:

```
>> echo hello
hello
>> echo #hello

>>
```



Permissions

- Ownership
- chmod
- chown

Ownership

- Linux supports a large number of users, consequently it needs to keep track of who and how users can access files and folders.
- These rules are called permissions.
- There are three common types of users with permissions.
  - User
    - These apply to a single user, it gives them special access. The user is the '**owner**' of the file.
  - Group
    - Apply to a single group of users; the '**owning group**'.
  - Other
    - All other users or '**others**' aka '**the world**'.

https://www.computerhope.com/unix/uchown.htm

Ownership

- A common way to define permissions is with a numerical representation of what each of the three groups can do:

Value	How the value is calculated	Symbolic	Permission
7	4(r) + 2(w) + 1(x)	rwx	read, write and execute
6	4(r) + 2(w)	rw-	read and write
5	4(r) + 1(x)	r-x	read and execute
4	4(r)	r--	read only
3	2(w) + 1(x)	-wx	write and execute
2	2(w)	-w-	write only
1	1(x)	--x	execute only
0		---	none

chmod

- https://ss64.com/bash/chmod.html
- Change access permissions, **ch**ange **mod**e.
- Supports ‘numeric mode’ and ‘symbolic mode’

`chmod` *`nnn file`*

e.g. `chmod 774 myscript.sh`

This gives the myscript.sh file read, write and execute for User and Group, but only read for Other.

chown

- https://ss64.com/bash/chown.html

- Change owner, change the user and/or group ownership of each given File to a new Owner.

```
chown filename [user[:group]]
```

e.g.

```
groupadd new_group
ls -l
chown hello.c root:new_group
ls -l
```

```
root@tryit-holy:~# groupadd new_group
root@tryit-holy:~# ls -l
total 0
-rw-r--r-- 1 root root 0 Mar  4 14:19 hello.c
root@tryit-holy:~# chown root:new_group hello.c
root@tryit-holy:~# ls -l
total 0
-rw-r--r-- 1 root new_group 0 Mar  4 14:19 hello.c
root@tryit-holy:~#
```

Redirection

- Directing data:
  - *command* < *file*
  - *command* < (*command*)
  - *command* > *file*
  - *command* >> *file*
  - *command* | *command*

I/O Redirection



http://bit.ly/2VqsGDj

- There are always three default files open:
  - **stdin** (the keyboard, channel 0)
  - **stdout** (the screen, channel 1)
  - **stderr** (error messages output to the screen, channel 2).

- These, and any other open files, can be redirected.

- Redirection simply means capturing output from a file, command, program, script, or even code block within a script and sending it as input to another file, command, program, or script.

https://www.tldp.org/LDP/abs/html/io-redirection.html

[The Linux Documentation Project tldp.org]

Overwriting COMMAND_OUTPUT >

- Redirect **stdout** to a file.
  - Creates the file if not present, otherwise overwrites it.

```
ls -lR > dir-tree.list
```

Appending COMMAND_OUTPUT >>

- Redirect **stdout** to a file.
  - Creates the file if not present, otherwise appends to it.

```
ls -lR >> dir-tree.list
```

< FILENAME

< FILENAME

- Accept input from a file.
  - Companion command to ">", and often used in combination with it.

e.g.

```
grep *.c < dir-tree.list
```

```
[root@localhost ~]# grep *.c < dir-tree.list
-rw-r--r--    1 root     root           185 Sep  9 13:26 hello.c
-rw-r--r--    1 root     root           185 Sep  9 13:26 hello.c
[root@localhost ~]#
```

The Pipe "|"

- Pipe "|" (represented by the vertical bar) is used send output to another program.

e.g.

```
cat | sort > sortdata.txt
line 1
line 2
line 0
<CTRL> + D
clear
cat sortdata.txt
```

```
[root@localhost ~]# cat sortdata.txt
line 0
line 1
line 2
[root@localhost ~]#
```

Goals for Today

- Directory commands: ls, cd, mkdir, rmdir and pwd
- File commands: touch, cat, cp, mv, rm, head and less
- Writing to stdout with echo, and commenting your commands
- File permissions and ownership, and changing them with chmod and chown
- Redirection with >, >> and <, and joining commands with the pipe |

CS-107 Lecture 20
Working with Files

Troy Astarte

Megan Venn-Wycherley